

Monitoring Processes in Linux



Andrew Mallett

Author and Trainer

@theurbanpenguin www.theurbanpenguin.com



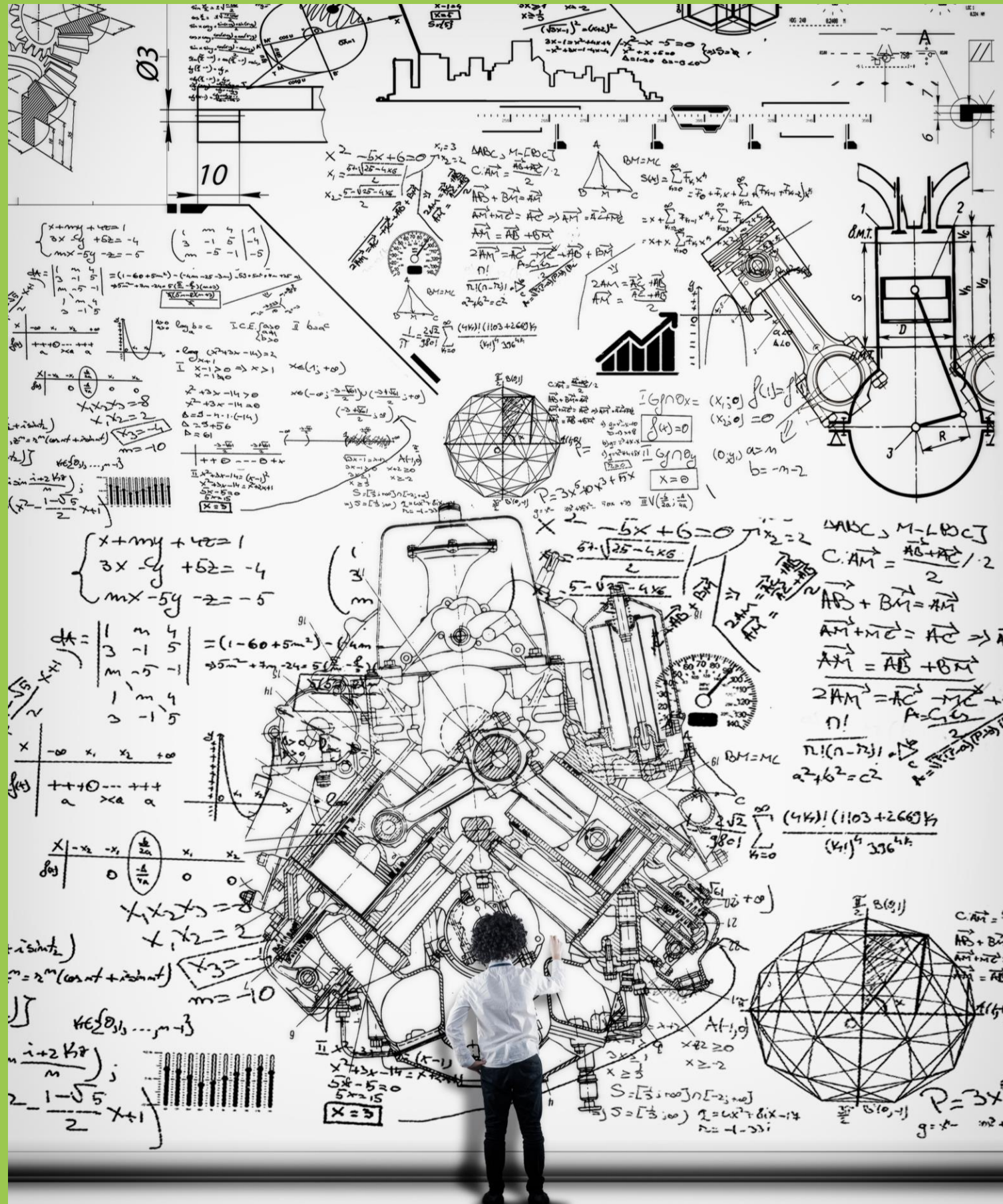
Objectives



Monitoring Processes and Performance

- Diagnose Ubuntu multipathd
- View processes by CPU and memory utilization
- List process status
- Share CPU with nice and renice
- Search and end processes





We Have a Problem

Detecting and resolving issues is always a useful skill. We have seen previously, that the Ubuntu Multipath daemon is misconfigured. Let's fix it.



```
$ sudo journalctl --unit multipathd -f  
$ sudo systemctl status multipathd  
$ man 5 multipath.conf
```

Multipathd

The multipath daemon allows for multiple network paths and topologies to be used to connect to storage arrays. The local drives `/dev/sda` and `/dev/sdb` have not been excluded from topology mapping and, as a result, produce vast amounts of errors.



```
- name: correct multipathd
  copy:
    dest: /etc/multipath.conf
    content: |
      defaults {
        user_friendly_names yes
      }
      blacklist {
        devnode "^sd[a-b]"
      }
  notify: RestartMP
```

Ansible Provisioner with Vagrant

The `ansible_local` provisioner in Vagrant will install Ansible into the guest and then can execute a Playbook. This way we can ensure each time the virtual machine is created the fix is applied.



Demo



Working on Ubuntu1

- investigate multipathd
- correct using Playbook
- adjust Vagrantfile



```
$ nproc
```

```
$ uptime (1min) (5min) (15min)
```

Uptime and Load Averages

The command uptime not only shows how long the system has been up for, but the CPU load averages. To calculate the CPU utilization, divide by the number of processors.



Demo



CPU Utilization

- counting processors
- calculation load averages



```
$ top
```

Using Top

The command `top` summarizes the system performance and sorts processes by CPU utilization but can be changed by using the `f` key



Demo



Working with Top

- using top
- sorting by memory usage



```
$ ps  
$ ps -f  
$ ps -fl  
$ ps -elf  
$ ps -lfp 1
```

Using Ps (Process Status)

To list processes, we make use of the ps command, all running processes exists as subdirectories below /proc



```
$ ps -elf | grep bash  
$ pgrep bash
```

Pgrep

Using pgrep we can combine ps and grep to search for processes



```
$ sleep 1000 &  
$ pkill sleep  
$ kill -1
```

Ending Processes

Rather than searching for processes and ending them as required with kill, we can combine the actions using pkill. The default signal is to terminate but other signals can be sent as required.



Demo



Listing and Ending Processes

- ps
- pgrep
- pkill



```
$ sleep 1000&  
$ ps -lfp $(pgrep sleep)  
$ nice -n10 sleep 1000&  
$ ps -lfp $(pgrep sleep)  
$ renice -n 19 -p <PID>  
$ renice -n 15 -p <PID>
```

Sharing CPU

The nice value of a process determines the CPU priority that the process will get, ranging from -20 to +19. Running processes can be reniced to nicer values by standard uses but not to less nice values.



Demo



Process Priority

- nice
- renice



Git Repo

```
git clone https://github.com/theurbanpenguin/lfcs
```



Summary



Monitoring Linux

- correcting multipathd
- uptime
- nproc
- ps
- pgrep
- pkill



Up Next:
Securing Ubuntu Using AppArmor

